Faculty Office-Hours Scheduler

A Web Platform for Structured Faculty–Student Appointment Booking

Course Project Proposal — UCS503P

Submitted to: Dr. Raghav B. Venkataramaiyer
Department of Computer Science & Engineering (COPC)
Thapar Institute of Engineering and Technology, Patiala

Student Team (5th Semester B.Tech COE/COPC)

Name	Roll Number
Manan Garg	1024170227
Vishesh	1024270355
Dravinam	1024170244
Aryan	1024170240

August 21, 2026

1. Introduction

Getting fifteen minutes of a professor's time outside class hours is one of those things every student needs at some point — before a quiz, while stuck on a project, or just to get feedback on an assignment — but it rarely works out cleanly. At Thapar, this usually means one of two things: standing outside a faculty cabin hoping they're free, or sending an email and waiting a day or two for a reply that may or may not land at a time you're actually free. Both approaches waste time on both sides.

This proposal describes a web-based Faculty Office-Hours Scheduler that lets faculty members and TAs publish their available hours, and lets students search and book a slot directly — the same way one might book a badminton court on a campus sports app. The system is built around three priorities that guided every design decision below: it has to be genuinely simple for faculty to adopt (nobody wants to learn a new tool that takes longer than just replying to an email), it has to guarantee that two students can never end up booked into the same slot, and it has to actually get used, not sit as an unused feature after week one.

Sections 2–4 lay out the problem and what already exists in this space; Sections 5–7 describe the system itself and how we plan to build it; and Sections 8–11 cover how we plan to evaluate it, what we'll deliver at each stage, and how we're managing risk over the six-week build.

2. Problem Statement & Motivation

During a regular semester, the informal way office hours currently work creates a handful of recurring problems that most of us have run into personally:

- **No real-time visibility into availability:** there is no way to check whether a professor is actually in their cabin right now without walking there. Students often make multiple trips across campus, sometimes between different academic blocks, only to find the door closed or a class running late.
- **Overcrowding at some hours, empty slots at others:** because there is no coordination, a large group of students often shows up at the same informally-known time, typically right before a deadline, while the professor's officially free hours on other days go completely unused.
- **No record of who booked what:** without a shared calendar, two students can be told the same time slot by mistake, or a meeting can simply be forgotten by either side. There is also no way to look back at how office hours were actually used over a semester.
- **Coordination happens over scattered channels:** email threads, WhatsApp groups, and one-off Google Forms circulated by CRs all get used inconsistently, which means faculty end up manually replying to some version of "are you free at X?" dozens of times a semester.

None of these are new problems — they show up in some form at almost every university — which is also part of why we looked closely at what already exists before deciding to build something from scratch (Section 4).

3. Objectives

- Provide a single, always-current source of truth for office-hour availability, searchable by course, faculty name, or day.
- Guarantee, at the database level and not just in the UI, that no slot can ever be double-booked.
- Keep the admin overhead for faculty down to a few clicks a week, so adoption does not depend on people changing their habits.
- Give department coordinators an honest picture of how office hours are actually used — utilization and no-show rate — instead of relying on anecdotes.
- Keep the whole system deployable on free-tier infrastructure so it can realistically run for a semester-long pilot without needing institute IT to provision anything.

4. Existing Solutions, and Why We Are Not Just Using Them

It's a fair question — tools like Calendly, Google Calendar's appointment slots, and Doodle already solve the general "let people book a meeting with me" problem reasonably well, and we did seriously consider wiring one of these up instead of building a new system. A few reasons we decided against it:

- **No concept of course or roll number:** none of these tools treat "course" or "roll number" as first-class data, so there is no way to filter slots by subject or automatically tag a booking with the student's course and section — which matters both for search and for the analytics we want to show department coordinators.
- **Free-tier limits don't scale to a department:** Calendly, for instance, caps free accounts at a single active event type per user, and paying per faculty member is not realistic for a student-run pilot.
- **No attendance or no-show tracking:** at best these tools give you a calendar entry, not a record of whether the meeting actually happened, which rules out the no-show metric that is central to our evaluation plan (Section 8).
- **Different concurrency profile:** dozens of students from the same course hitting "book" within seconds of each other, right before a deadline, is a fairly different load pattern than the one-off 1:1 meetings these tools are optimized for.

Building our own also meant we could apply what we've studied this semester in DBMS and OS directly — concurrency control and transaction isolation are exactly the kind of problem this project needed, and it felt more useful than gluing together a third-party tool we couldn't customize.

5. Proposed System Architecture

5.1 High-Level Architecture & Modules

The platform is organised as a responsive single-page web application split into five modules:

- **Faculty Portal:** a dashboard where faculty and TAs add single or recurring weekly office-hour slots through a simple calendar picker — pick a weekday, a start/end time, and optionally a repeat rule and an end date for the recurrence. Existing slots can be edited or withdrawn (a slot can only be withdrawn if it has not already been booked).
- **Student Portal:** a searchable slot list that students filter by course code, faculty name, or day, and book instantly. The list re-fetches automatically (via periodic polling in the first version, see Section 6.2) so a slot that just got taken by someone else disappears without the student needing to refresh manually.
- **Conflict Prevention Engine:** server-side logic and database constraints that make double-booking physically impossible rather than just unlikely — this is described in detail in Section 6.1, since it is the piece we spent the most design time on.
- **Notification System:** automated email confirmations sent the moment a booking is made, a reminder email dispatched shortly before the session, and a prompt to the faculty member after the session asking them to log the outcome.
- **Analytics Dashboard:** aggregate, read-only metrics for faculty and admins — total bookings, slot utilization percentage, and no-show statistics per course and per faculty member, computed with simple grouped SQL queries rather than a separate analytics pipeline.

5.2 Core Workflow

- Slot creation: a faculty member logs in and defines open office hours for the week, e.g. Tuesday 3:00–4:30 PM, either as a one-off or a recurring weekly rule.
- Search & booking: a student searches by course code or faculty name, views open slots, selects one, and enters a short note describing what they want to discuss (this note is shown to the faculty member ahead of time, so they can come prepared).
- Confirmation & reminder: the system confirms the booking instantly in the UI and by email, and sends a reminder email to both parties before the session starts.
- Cancellation / rescheduling: either side can cancel up to a configurable cutoff (default 2 hours) before the slot; a cancelled slot is immediately reopened for other students to book, rather than sitting reserved and unused.
- Post-session log: after the meeting, the faculty member marks the slot as Completed or No-Show, with an optional one-line note — this is the data point the no-show metric in Section 8 is built on.

5.3 Data Model & Database Design

The schema is intentionally small — six tables — since the goal is correctness and clarity, not flexibility we don't need yet:

Table	Purpose
users	Student, faculty and admin accounts (name, institute email, roll no. / employee id, hashed password, role).
courses	Course code and title (e.g. UCS405 — Discrete Mathematical Structures).

Table	Purpose
faculty_courses	Many-to-many map between faculty/TAs and the courses they take office hours for.
office_hour_slots	A single bookable time window: faculty_id, start_time, end_time, recurrence rule (if any), location, status.
bookings	A student's reservation against a slot: student_id, course_id, query note, status (booked / completed / no_show / cancelled), timestamps.
notifications	Log of emails sent per booking — type (confirmation / reminder), channel, sent_at — mainly for debugging delivery issues during the pilot.

The constraint that matters most is on office_hour_slots, since it's what makes double-booking structurally impossible rather than just something the application layer tries to prevent:

```
CREATE TABLE office_hour_slots (
  id              SERIAL PRIMARY KEY,
  faculty_id      INTEGER REFERENCES users(id),
  start_time      TIMESTAMPTZ NOT NULL,
  end_time        TIMESTAMPTZ NOT NULL,
  status          TEXT NOT NULL DEFAULT 'open', -- open | booked | cancelled
  UNIQUE (faculty_id, start_time)
);
```

5.4 Design Considerations for Campus Deployment

- **Mobile-first responsiveness:** most students will open this from a phone between classes, so the UI is built mobile-first and tested on the kind of low-bandwidth Wi-Fi you get on campus, not just on a laptop on hostel Wi-Fi.
- **System reliability over extra features:** we're deliberately prioritising transactional safety and clean state handling — a booking flow that never breaks — over adding features that look good in a demo but aren't load-bearing.
- **Easy, free hosting:** the entire stack runs on free/hobby-tier hosting (Vercel, Render, and a managed Postgres instance) so the pilot doesn't depend on getting institute infrastructure approved mid-semester.

6. Technical Implementation Details

6.1 Concurrency Handling & Database Integrity

The scenario we designed around: two students, on the same course WhatsApp group, see a slot go up and both hit "Book" within the same second, right before an assignment deadline. If the check-then-insert happens naively in application code, both requests can read the slot as "open" before either write lands, and both bookings succeed — the exact bug this project exists to prevent. We handle this at two levels:

- **Database composite unique index:** a UNIQUE constraint on (faculty_id, start_time) means the database itself physically rejects a second slot at the same time for the same faculty member, regardless of what the application code does.
- **Atomic booking transactions with row locking:** a booking is wrapped in a single SQL transaction that locks the target slot row (SELECT ... FOR UPDATE), checks its status is still 'open', inserts the booking, and flips the slot's status to 'booked' — all inside one BEGIN...COMMIT block, so a second concurrent request simply waits for the lock and then sees the slot as already booked, instead of racing past the check.

```
BEGIN;
SELECT status FROM office_hour_slots WHERE id = $1 FOR UPDATE;
-- application checks status = 'open', else ROLLBACK and return 409 Conflict
INSERT INTO bookings (slot_id, student_id, course_id, note, status)
  VALUES ($1, $2, $3, $4, 'booked');
UPDATE office_hour_slots SET status = 'booked' WHERE id = $1;
COMMIT;
```

- **Server-side slot expansion:** recurring rules like "every Thursday, 2–4 PM, for the next 8 weeks" are expanded into concrete individual slot rows by a backend job at creation time, rather than being interpreted on the fly — this keeps the uniqueness constraint and booking logic above simple, since every bookable slot is always a real row in the table.

We plan to verify this with a small load test before the pilot — firing 20–30 concurrent booking requests at the same slot and confirming exactly one succeeds — rather than assuming the constraint works and finding out otherwise during the pilot.

6.2 Technology Stack & Justification

Layer	Technology	Why
Frontend	React.js + Tailwind CSS	Component-based UI the whole team already has experience with from earlier coursework; Tailwind keeps styling consistent without building a separate design system.
Data fetching	TanStack (React) Query	Slots can be booked by someone else while a student is browsing, so we need frequent re-fetching / refetch-on-focus rather than data that goes stale silently.
Backend API	Node.js + Express.js	Keeps the whole stack in JavaScript so team members can move between frontend and backend work without a language switch, and matches what we've used in lab assignments.
Database	PostgreSQL	Relational integrity, transactions, and row-level locking are core requirements here (Section 6.1) — a NoSQL store would just mean re-implementing what Postgres already gives us.

Layer	Technology	Why
Auth	JWT (access + refresh tokens), bcrypt for password hashing	Stateless auth that's simple to reason about at this scale; the role claim embedded in the token drives student / faculty / admin route access.
Email	Nodemailer via a transactional email provider (e.g. Brevo / Resend free tier)	Sending directly through a personal Gmail SMTP account is unreliable at any real volume and gets flagged as spam quickly; a transactional provider gives delivery logs we can debug from.
Hosting	Vercel (frontend), Render (API + cron), Neon/Supabase (managed Postgres)	All three offer usable free tiers, auto-deploy from GitHub on push, and don't require us to manage a server ourselves.

We briefly considered Firebase for a faster start, but decided against it — the strict relational constraints and multi-row transactions in Section 6.1 are much more natural in Postgres than in a document store, and getting that right matters more here than saving setup time.

6.3 API Design

The REST API surface is kept deliberately small for the MVP:

Method	Endpoint	Purpose
POST	/api/auth/register	Create a student or faculty account (restricted to institute email domain).
POST	/api/auth/login	Authenticate and issue a JWT access/refresh pair.
GET	/api/slots	Search open slots by course, faculty, and/or date range.
POST	/api/slots	Faculty creates a one-off slot or a recurring rule.
POST	/api/bookings	Student books a slot (runs inside the transaction in Section 6.1).
PATCH	/api/bookings/:id/cancel	Cancel a booking; the underlying slot reopens automatically.
PATCH	/api/bookings/:id/status	Faculty marks a past booking Completed or No-Show.
GET	/api/analytics/summary	Aggregate utilization and no-show numbers for the dashboard.

6.4 Testing Strategy & CI/CD Pipeline

- **Unit tests:** Jest for backend route handlers and booking logic (including the race-condition case above, using a test database transaction that's rolled back after each test); React Testing Library for frontend components.
- **Integration tests:** Supertest-driven API tests that hit a disposable test Postgres instance, covering the full booking → cancel → rebook cycle end to end, not just individual functions.

- **Continuous integration:** a GitHub Actions workflow runs on every push and pull request: install dependencies, run ESLint, run the Jest suite, and build the frontend — a PR cannot be merged into main if any step fails.
- **Continuous deployment:** a passing build on main auto-deploys the backend to Render and the frontend to Vercel, so the staging environment always reflects the latest merged code without anyone running a manual deploy step.

We're also keeping a short Postman/Newman collection for manually exercising the API during development, mainly so bugs get caught before they reach the automated suite.

7. Development Strategy & Timeline

We're following an iterative, agile plan across six weeks, tracked on a GitHub Projects board with weekly check-ins so we catch slippage early rather than at the end. The original three two-week phases break down as follows:

Week	Focus	Key tasks
1	Setup & auth	Repo scaffold, DB schema + migrations, JWT auth, base layout for both portals.
2	Core booking loop	Faculty slot-creation UI, student search & booking UI, transactional booking logic (Sec. 6.1).
3	Staging deploy & internal testing	Deploy to Render/Vercel staging, walk through the full booking flow end to end, fix UI bugs, start recruiting pilot faculty.
4	Pilot launch & reminders	Begin the 2-week pilot with 3–5 faculty/TAs and 20–30 students; ship Nodemailer confirmation and reminder emails.
5	Recurring slots & attendance	Recurring-rule expansion job, Completed/No-Show marking UI, mid-pilot feedback check-in.
6	Analytics & wrap-up	Analytics dashboard, close out pilot feedback collection, final bug fixing, demo and report preparation.

8. Evaluation Metrics & Pilot Plan

8.1 Primary Evaluation Metric

No-show rate: the percentage of confirmed bookings where the student or faculty member did not attend, calculated directly from the session outcome status logged by the instructor after each meeting. Based on informal conversations with a few TAs about how often students skip informally-agreed meetings, we're estimating the current baseline is somewhere around 30–40%; our target for the pilot is to bring the tracked no-show rate under 15% purely through the confirmation + reminder emails, without adding any penalty or enforcement mechanism.

8.2 Secondary Evaluation Metrics

Metric	Definition	Target
Average wait time	Time between a student opening the search view and the earliest bookable slot for their target faculty member.	Tracked per search; used as a baseline, no fixed target for the pilot.
Slot utilization rate	Percentage of published office-hour slots that get successfully booked.	≥ 60% during the pilot window.
Booking friction	Clicks and time required for a student to go from search to a confirmed booking.	≤ 4 clicks, median under 30 seconds.
Platform uptime	Share of the pilot period the app responds successfully to requests.	≥ 99% during regular campus hours.

8.3 Pilot Testing Plan

- Participants: a 2-week trial with 3–5 faculty members/TAs and roughly 20–30 students drawn from active lab sections, recruited directly rather than through a broad announcement, so we get consistent usage to measure against.
- Baseline comparison: before the pilot starts, we'll do a quick informal survey of participating faculty and students about how office hours currently work (how often meetings get missed, how long it takes to arrange one) to have something concrete to compare the pilot metrics against, rather than comparing to a guess.
- Feedback collection: a short Google Form after the pilot, plus a couple of informal follow-up conversations with participating faculty, focused on friction points the analytics dashboard wouldn't show on its own.
- Success criteria: we'll consider the pilot a success if the no-show rate drops meaningfully against the informal baseline and slot utilization clears 60%, since both indicate the tool is displacing the old manual process rather than sitting alongside it unused.

9. Project Deliverables

9.1 Phase 1 — Basic MVP (Weeks 1–3)

- Faculty dashboard to add open time slots manually.
- Student search view to browse available slots by professor/course and reserve one.
- Database constraints and atomic transactions preventing double-bookings.
- Initial GitHub Actions pipeline running backend automated unit tests.

9.2 Phase 2 — Full System (Weeks 4–6)

- Support for weekly recurring slot generation rules.
- Automated email confirmations and reminders sent via Nodemailer prior to scheduled meetings.

- Attendance tracking (Completed / No-Show) with optional student query notes and cancellation handling.
- Faculty and admin analytics dashboard showing slot utilization and attendance statistics.

9.3 Phase 3 — Stretch Goals (if time permits beyond Week 6)

These are explicitly not committed scope, but are natural next steps if the pilot goes well and we want to keep the project going past this course:

- One-way Google Calendar / ICS export so a booked slot shows up on a student's or faculty member's personal calendar.
- Admin panel for managing course-to-faculty mappings without touching the database directly.
- SMS reminders (via a service like Twilio) as a backup channel for students who don't check email regularly.

10. Team Roles & Responsibilities

A rough first split of ownership based on everyone's strengths from past coursework — we expect this to shift a bit once we're actually building, especially around Weeks 4–5 when everyone will need to help with pilot support and bug fixes regardless of area.

Name	Roll No.	Primary Focus
Manan Garg	1024170227	Backend & database — Express API, schema design, the booking transaction logic in Section 6.1.
Vishesh	1024270355	Frontend — React components, Tailwind UI, both the faculty and student portals.
Dravinam	1024170244	DevOps & QA — CI/CD pipeline, Jest/RTL test suites, staging deployment and monitoring.
Aryan	1024170240	Integration — API integration between frontend and backend, the Nodemailer notification system, analytics queries, and documentation.

11. Risks & Mitigation Strategies

Risk	Mitigation
Low faculty adoption	Keep slot creation to under 3 clicks; start the pilot with enthusiastic TAs who already run high-volume doubt sessions, rather than a broad faculty rollout.
Slot availability shortage	Recruit TAs/faculty who already hold frequent office hours so the platform has real activity from day one instead of sitting empty.
Concurrency collisions	PostgreSQL unique index plus atomic transactions (Section 6.1), verified with a multi-user stress test before the pilot starts.

Risk	Mitigation
Data security & privacy	Store only what's needed — name, roll number, course ID, slot timing — hash passwords with bcrypt, and secure all endpoints with JWT authorization over HTTPS.
Email deliverability	Use a transactional email provider instead of raw SMTP, and monitor bounce/spam rates during the pilot so a delivery problem doesn't quietly undercut the reminder feature.
Timeline pressure from overlapping exams	Keep one buffer day per week in the schedule, and treat Phase 3 stretch goals (Section 9.3) as the first thing to drop if we fall behind, not the core MVP.

12. Summary

This project proposes a deployable web platform to cut down the coordination friction between students and faculty around office hours. It stays within the course's expectations by working in rapid weekly iterations, using continuous integration and deployment to support frequent, safe releases, and by evaluating itself against clear, measurable criteria — primarily the no-show rate, alongside slot wait time and overall utilization — rather than just shipping features and assuming they help.

More than anything, this is a chance to put concepts from our DBMS and OS coursework — transactions, isolation, concurrency control — into something that has to actually hold up under real, simultaneous use, rather than just pass a lab exercise. If the pilot goes the way we expect, the end goal is a small, boring, reliable tool that quietly removes one recurring source of friction on campus — which, for a system like this, is exactly the point.